

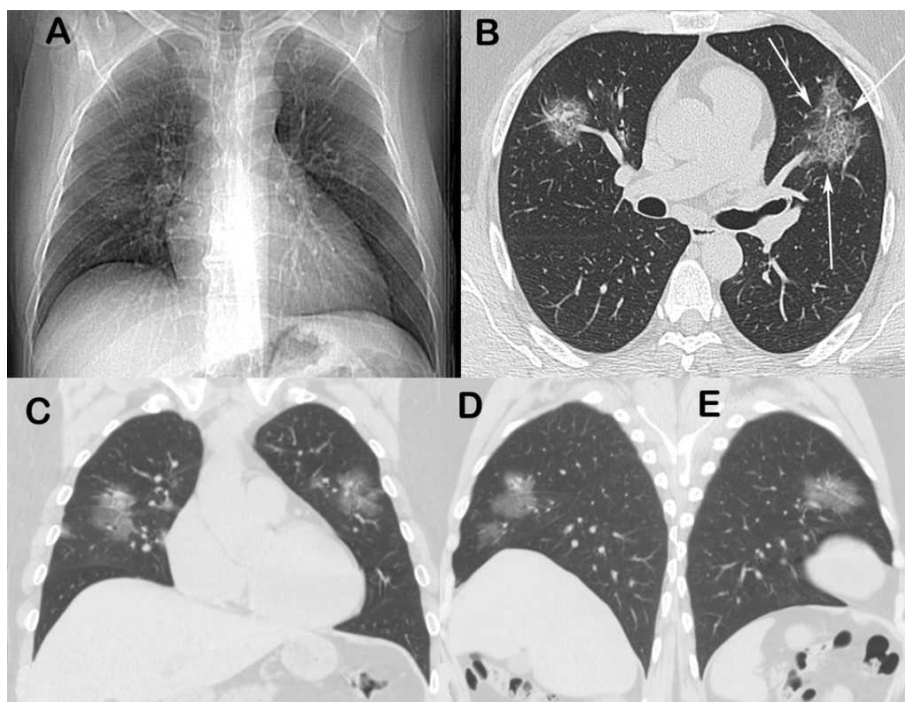
Binarna klasifikacija resnosti COVID-19 okužbe s pomočjo CT slik prsnega koša

5. december 2025

1 Uvod

Pandemija koronavirusne bolezni COVID-19 je v svojem vrhuncu močno obremenila celotni zdravstveni sistem [1, 2]. Takrat bi natančne napovedi resnosti okužbe s COVID-19 lahko znatno pomagale pri odločitvah o hospitalizaciji in zdravljenju [3].

Računalniška tomografija prsnega koša (CT) je neprecenljivo orodje za oceno pljučne prizadetosti [4, 5, 6]. Pri pljučnici, ki nastane zaradi okužbe s COVID-19, so najpogostejše kvalitativne CT značilnosti predvsem prisotnost mlečnega stekla (ang. *Ground Glass Opacities - GGO*) in konsolidacije (zgostitve) (Slika 1) [4].



Pulmonol 10.1016/j.pulmoe.2020.04.001

Slika 1: Primeri mlečnega stekla. Povzeto iz [13].

Z napredovanjem bolezni se načeloma povečuje tudi prizadetost pljučnega parenhima; pojavita se lahko tudi t.i. vzorec *norega* tlakovanja (ang. *crazy paving*) in obrnjenega *halo* znaka (ang. *reversed halo sign*) [4, 6] (Slika 2). Radiologi CT slike pljuč ocenjujejo kvalitativno, sama ocena pa se je izkazala kot koristna pri diagnozi [5, 7, 8] in določanju resnosti COVID-19 okužbe [6, 9, 10, 11].



(a) Obrnjeni halo znak.

(b) Vzorec norega tlakovanja.

Slika 2: Napredovanje bolezni poveča prizadetost pljuč. Povzeto iz [13].

Problem z ročnim ocenjevanjem je predvsem v tem, da je odkrivanje subtilnih kvalitativnih CT značilnosti zelo zahtevno, ocene so subjektivne, delo pa dolgotrajno.

Ali nam pri tem lahko pomaga umetna inteligenca? V tej nalogi se boste spoznali s konvolucijskimi nevronskimi mrežami in poskusili klasificirati paciente glede na resnost COVID-19 okužbe.

2 Računalniška tomografija

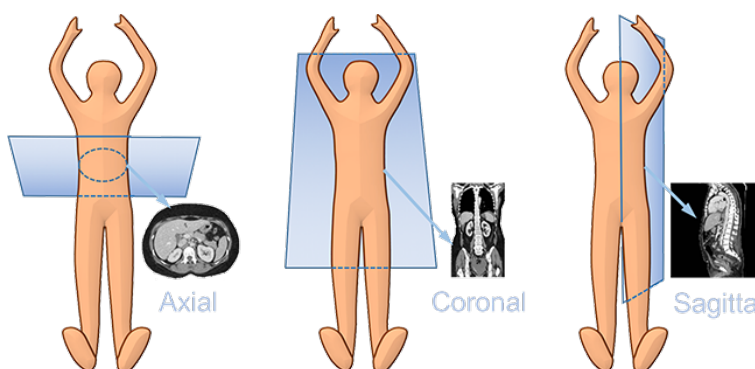
Tomografija je skupno ime za slikovne metode, pri katerih prikažemo le določen prerez človeškega telesa; *tomo* (grška beseda za rezino) in *graphos* (grška beseda za sliko).

Med tomografske metode prištevamo klasično oz. geometrijsko tomografijo, tomosintezo in računalniško tomografijo (ang. *Computed Tomography - CT*) (Slika 3).



Slika 3: Prikaz CT naprave Siemens. Povzeto iz [14]

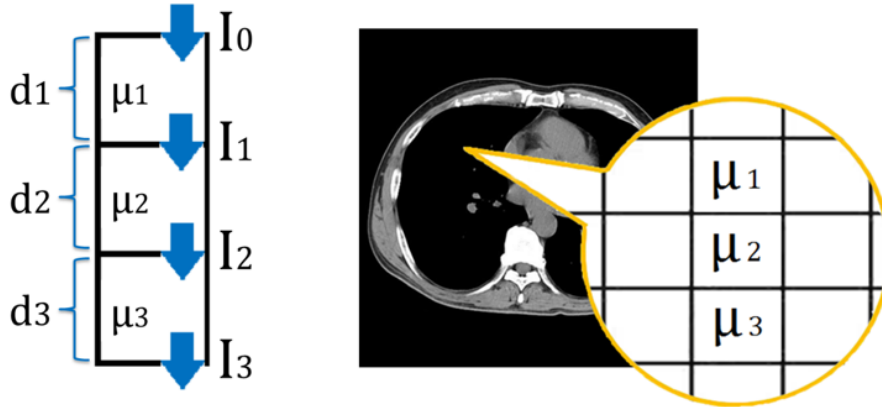
Računalniška tomografija je večstopenjski proces: skeniranje, rekonstrukcija in predstavitev zbranih podatkov. Omogoča prikaz prerezov človeškega telesa z zelo visoko kontrastno ločljivostjo in zato dobro preglednostjo tkiv, ki imajo majhen lastni kontrast (Slika 4). Cena, ki jo pri tem plačamo, pa je visoka izpostavljenost pacientov v primerjavi s konvencionalno projekcijsko rentgenografijo. Po ocenah strokovnjakov Evropske skupnosti, danes računalniška tomografija prispeva skoraj že polovico h kolektivni dozi zaradi diagnostične radiologije [16].



Slika 4: Prikaz aksialne, koronarne in sagitalne ravnine. Povzeto iz [15].

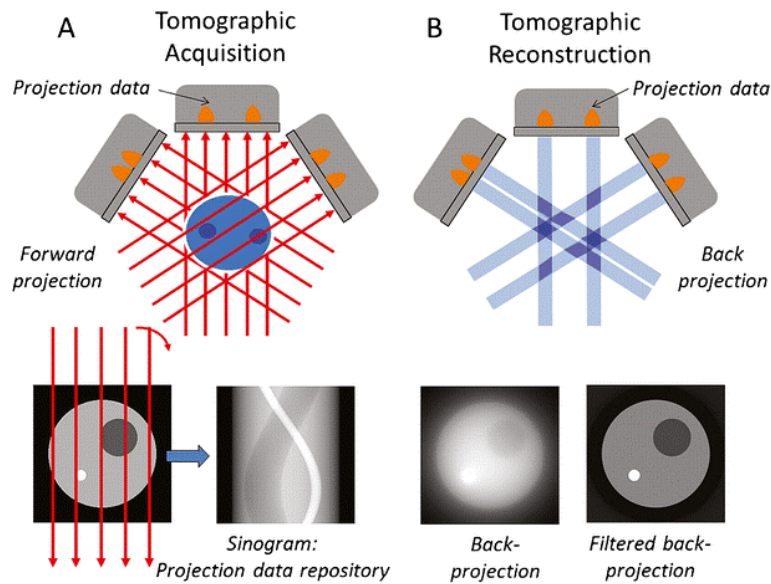
2.1 Proces pridobivanja slike

Predstavljajmo si, da imamo zaprto škatlo. Obsevamo jo iz večih strani in izmerimo, kaj pride skozi. Na ta način bi morali ugotoviti strukturo njene notranjosti. Slikano področje razdelimo na kocke z enako prostornino, ki jih imenujemo voksli (osnovni prostorninski element). Cilj je ugotoviti atenuacijske koeficiente po vseh vokslih. Ko sevanje prehaja skozi snov, prihaja do atenuacije. Po prehodu snovi debeline $h = d1 + d2 + d3$ (Slika 5), se začetna jakost sevanja I_0 oslabi na $I = I_0 e^{-uh}$, kar izmerimo z detektorjem na drugi strani. Če stolpec višine h razdelimo na enake dele ($d1 = d2 = d3 = d$), lahko zapišemo $I = I_0 e^{-(u1+u2+u3)d}$. Za rešitev danega preprostega sistema potrebujemo vsaj 3 enačbe.



Slika 5: Atenuacijski koeficienti pri prehajanju rezin. Povzeto iz [16].

Ideja je torej v tem, da iz dovolj velikega števila projekcij dobimo dovolj enačb (Slika 6), rešitev katerih so vrednosti atenuacijskih koeficientov po vseh vokslih.



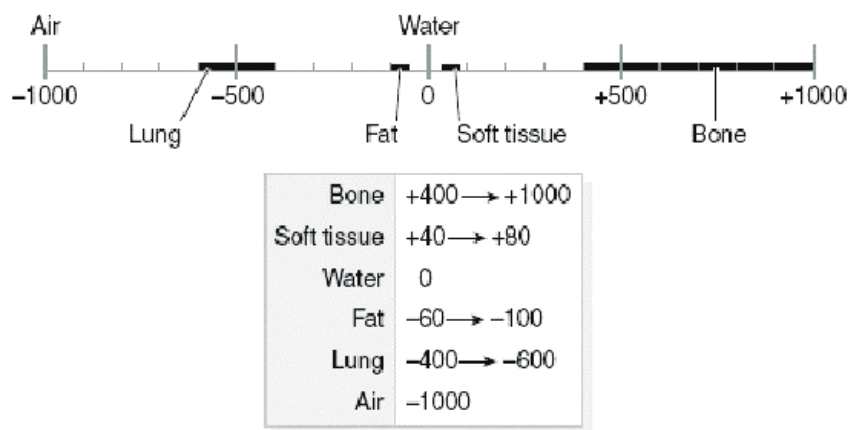
Slika 6: Predstavitev ideje poteka računalniške tomografije. Povzeto iz [16].

2.2 CT števila

Vsakemu prostorninskemu elementu priredimo vrednost CT števila, ki ustreza linearnemu oslabitvenemu koeficientu posameznega prostorninskega elementa. Za predstavitev CT števil se v medicini uporabljajo Hounsfieldove enote, in sicer

$$HU = 1000 \cdot \frac{\mu - \mu_{voda}}{\mu_{voda} - \mu_{zrak}}, \quad (1)$$

pri čemer je μ atenuacijski koeficient za izbrano tkivo, μ_{voda} in μ_{zrak} pa zaporedoma atenuacijska koeficienta vode in zraka. V medicinsko obarvani literaturi se velikokrat atenuacijskega koeficienta za zrak ne piše (vrednost skoraj nič), tako da je referenčna vrednost HU enot kar atenuacija vode. Tkiva, katerih atenuacijski koeficient je večji od atenuacijskega koeficienta vode imajo pozitivna CT števila, tkiva z manjšim pa negativna. Radiološke preiskave z računalniško tomografijo navadno potekajo pri visokih napetostih na rentgenski cevi (120 kV). Pri teh energijah fotonov v mehkih tkivih nad fotoefektom prevlada Comptonski pojav, kjer je atenuacijski koeficient določen predvsem z gostoto tkiva. Atenuacija je v tem primeru torej odvisna pretežno od gostote snovi, zato CT slika prikazuje gostoto tkiv (Slika 7).



Slika 7: CT števila za različna tkiva. Povzeto iz [15].

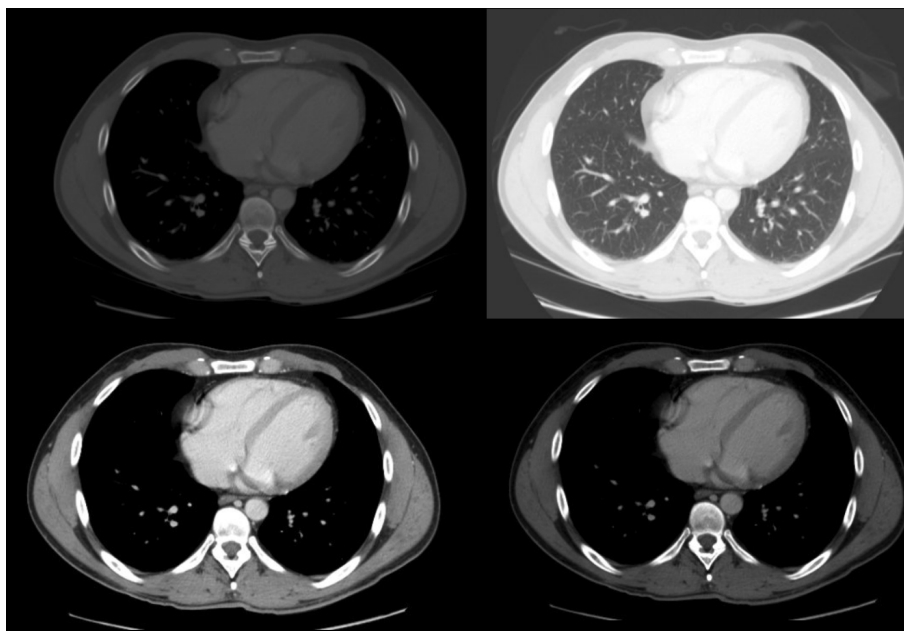
2.3 Predstavitev zbranih podatkov

Zvezo med CT števili in svetlostjo površinskih elementov na ekranu določa izbrano okno (ang. *windowing*), ki ga izbere izvajalec posega. To predstavlja interval CT števil, ki bodo zajele celotno skalo črnitve. S spreminjanjem najvišjih in najnižjih vrednosti CT števil okna kontroliramo raven kontrasta. Majhno okno bo predstavljalo velik kontrast, saj bodo majhne spremembe v CT številih predstavljene z večjimi spremembami črnitve. Veliko okno bo pomenilo nizek kontrast, hkrati pa bo omogočalo vidnost tkiv z velikimi razlikami v CT številih. Tkiva, ki imajo CT števila nižja od spodnje meje bodo na sliki črna, tista z višjimi vrednostmi pa bela. Tkiva s CT števili med zgornjo in spodnjo mejo bodo imela različne odtenke sive.

Za analizo različnih telesnih struktur se uporabljajo različna okna - W (ang. *window*) in nivoji - L (ang. *level*) sivine. Primer:

- Možgani: $(W, L) = (80, 40)$
- Pljuča: $(W, L) = (1500, -600)$ (Slika 8 desno zgoraj.)
- Kostni: $(W, L) = (1800, 400)$
- Jetra: $(W, L) = (150, 30)$

Različni nivoji sivine so tako omejeni na interval $(L - W/2, L + W/2)$.



Slika 8: Različna okna in nivoji sivine za aksialni prerez pljuč. Povzeto iz [13].

3 Podatki

V nalogi so na voljo prečiščeni (odstranjene slabo kvalitetne CT slike in zmerne okužbe) podatki iz ruske prosto-dostopne baze *MOSMED* [12]. Gre za anonimizirane CT slike prsnega koša, zajete med marcem in aprilom 2020 (prvi val epidemije). V našem vzorcu je 452 pacientov, moški - 42 %, ženske - 56 %, neznanih - 2 %. Starost pacientov je v intervalu med 18 in 97 let, s povprečno starostjo 47 let. V prečiščenih podatkih se nahaja 226 pacientov z **blago** (ang. *mild*) COVID-19 okužbo in 226 pacientov z **hudo** (ang. *severe*) COVID-19 okužbo. Resnost COVID-19 okužbe je bila ocenjena glede na ročno radiološko oceno CT znakov in klinične podatke.

3.1 Blaga okužba

Klinični znaki:

- $T < 38.0\text{ }^{\circ}\text{C}$ (telesna temperatura)
- $RR < 20/\text{min}$ (hitrost dihanja)
- $\text{SpO}_2 > 95\text{ }\%$ (periferna kapilarna nasičenost s kisikom)

CT znaki:

- Prisotnost mlečnega stekla (ang. *Ground Glass Opacities - GGO*)
- Prizadetost pljučnega parenhima (manjša od 25 %)

3.2 Resna okužba

Klinični znaki:

- $T \geq 38.5\text{ }^{\circ}\text{C}$ (telesna temperatura)
- $RR \geq 30/\text{min}$ (hitrost dihanja)
- $\text{SpO}_2 \leq 95\text{ }\%$ (periferna kapilarna nasičenost s kisikom)
- $\text{FiO}_2 \leq 300\text{ mmHg}$ (delež navdihnjenega kisika)

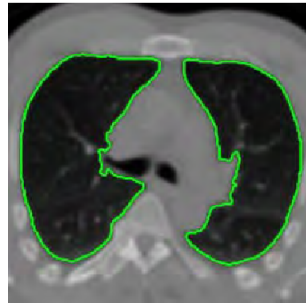
CT znaki:

- Prisotnost mlečnega stekla (ang. *Ground Glass Opacities - GGO*)
- Prizadetost pljučnega parenhima (večja od 50 %)
- Pulmonarne konsolidacije

Slike, objavljene v bazi *MOSMED* [12], so dimenzij $512 \times 512 \times N_z$. To pomeni, da je resolucija v aksialni ravnini 512×512 , vzdolž pacienta pa je N_z rezin. Objavljena je bila zgolj vsaka deseta rezina.

4 Segmentacija

Da se pri klasifikaciji resnosti COVID-19 okužbe izognemo nepotrebnemu šumu, je pljuča smiselno segmentirati. Z drugimi besedami to pomeni, da odstranimo irelevantne sosednje telesne strukture. Segmentacija pljuč iz CT slik, še posebej v aksialni ravnini, je zelo zanesljiva in relativno preprosta. V nalogi je ta problem že rešen. Prejeli boste binarne maske, ki so bile pridobljene s pomočjo Watershed algoritma (Slika 9).



Slika 9: Primer segmentacije pljuč z Watershed algoritmom.

5 Cilj naloge

Ker je treniranje modelov s konvolucijskimi nevronske mrežami lahko zelo dolgotrajno, je v zasnovi naloge narejenih nekaj kompromisov (velikost učne množice, izbira rezin).

V učni množici (60% vseh slik) so na voljo izbrane CT rezine v aksialnem pogledu, ki spadajo bodisi pod **blago** ali **resno** COVID-19 okužbo. Pri tem je zaradi zgoraj omenjenih razlogov, izbrana zgolj ena centralna CT-rezina na pacienta.

Poleg tega imate še od učne množice neodvisni validacijski (20%) in testni (20%) množici, pri čemer pa je za vsakega pacienta izbranih 10 centralnih aksialnih rezin (območje sredine pljuč, ki je za nas najbolj zanimivo).

Na danih CT rezinah v učni množici poskusite naučiti konvolucijsko nevronske mrežo tako, da bo pri testiranju zgolj iz 10 centralnih CT rezin čim boljše razlikovala med **blago** in **resno** COVID-19 okužbo. S pomočjo validacijske množice izberite najboljši model. Cilj je dobiti čim boljše rezultate na testni množici. Bolj natančna navodila so v poglavju Navodila naloge 8.

6 Predobdelava podatkov

Ta korak imate že narejen. Procesirane rezine slik se nahajajo v mapi `/data/PSUF_naloga/5-naloga/processed/`. Shranjene so v `.npy` formatu.

Za boljše razumevanje rezultatov, interpretacijo vaših modelov in vizualizacijo pa je zaželeno, da si ta del vseeno preberete.

6.1 Branje slike

Originalne CT slike so shranjene v formatu NIfTI-1 (`.nii`) in kompresirane z `.gz`. V datoteki ni shranjena zgolj informacija o CT sliki, ampak tudi metapodatki, kot so npr. debelina rezine, ime CT naprave, ime pacienta (odstranjeno zaradi anonimizacije), rekonstrukcijski algoritem ipd. Za branje teh datotek in obdelavo z vam domačim **numpy**-em, potrebujete knjižnico **nibabel**.

```
1 import nibabel as nib
2 import numpy as np
3 from matplotlib import pyplot as plt
4
5 filepath_CT = "data/raw/mild/study_0003.nii.gz" # pot do CT datoteke
6
7 all_image_info = nib.load(filepath_CT) # slika in vsi metapodatki
8
9 I_dataobject = all_image_info.dataobj # vzamemo le podatkovni objekt
10
11 I = np.array(I_dataobject) # pretvorimo v numpy matriko, dimenzij 512x512xNz
12
13 plt.imshow(I[:, :, 19]) # primer vizualizacije 20. rezine
```

6.2 Normalizacija slike

Dano sliko I sedaj normaliziramo. Za izbiro nivoja sivine uporabimo okno $W = 1500$ in nivo $L = -600$. Različni nivoji sivine so tako omejeni na interval $(L - W/2, L + W/2) = (-1350, 150)$.

```
1 def normalize(image, MIN_BOUND, MAX_BOUND):
2     # Normalizacija CT slike znotraj intervala MIN_BOUND, MAX_BOUND.
3     image = (image - MIN_BOUND) / (MAX_BOUND - MIN_BOUND)
4     image[image > 1] = 1.
5     image[image < 0] = 0.
6     return image
7
8 I = normalize(I, -1350, 150) # dobimo normalizirano sliko
```

6.3 Segmentacija slike

Maske so shranjene v formatu `.nrrd` (Nearly Raw Raster Data). Za branje boste potrebovali knjižnico **nrrd**. Maska je sestavljena iz binarnih elementov - torej iz 0 in 1, slednje predstavlja območje pljuč.

```
1 import nrrd
2
3 filepath_mask = "data/masks/mild/study_0003_mask.nrrd" # pot do maske
4
5 M, _ = nrrd.read(filepath_mask) # preberemo masko in jo shranimo v M
```

Ko imamo shranjeno normalizirano CT sliko v I in masko v M , to masko apliciramo na CT sliko, da dobimo segmentacijo S :

```
1 S = np.where(M==1, I, M)
```

6.4 Odstranitev praznih pikslov

Ko apliciramo masko na normalizirano CT sliko, bodo sama pljuča zasedala relativno majhno območje na sredini. Tega si ne želimo, zato poiščemo najmanjši možen pravokotnik, ki obdaja naša pljuča in jih raztegnemo na prvotno dimenzijo slike (alternativa je lahko t.i. *zero-padding*).

```
1 def bound_box_and_reshape(img, slice_idx):
2     # Za dano rezino poiščemo najmanjši pravokotnik, ki obdaja segmentirana pljuča.
3     I_slice = img[:, :, slice_idx]
4     rows = np.any(I_slice, axis=1)
5     cols = np.any(I_slice, axis=0)
6     rmin, rmax = np.where(rows)[0][[0, -1]]
7     cmin, cmax = np.where(cols)[0][[0, -1]]
8     I_slice = resize(I_slice[rmin:rmax, cmin:cmax], (512, 512))
9     I_slice = np.transpose(I_slice[:, :, np.newaxis], axes = [2, 0, 1]).astype('float32')
10
11     return I_slice
12
13 bounded_slice = bound_box_and_reshape(S, 9) # npr. da zelimo obrezati 10. rezino S-ja
```

7 Kratek uvod v okolje Pytorch

Skozi vse podrobnosti Pytorch-a, ki bodo potrebne za to nalogo, se bomo sprehodili na vajah. Tukaj je zgolj krajši uvod v katerem so predstavljene bistvene stvari. Z uporabo napisanih skript, ki so dostopne na učilnici in tega poglavja, bi vam moralo uspeti priti do rezultatov.

Koda je sestavljena iz 4 skript:

- `datareader.py` (branje podatkov)
- `model.py` (arhitektura konvolucijske nevronske mreže)
- `train.py` (učenje modela)
- `test.py` (testiranje modela)

7.1 Bralnik podatkov - DataReader

Bralnik podatkov se nahaja v skripti `datareader.py`, ki je ne rabite spreminjati. Lasten razred nabora podatkov - `DataReader` uporabljamo za enostavno in elegantno branje podatkov. Definirati moramo zgolj funkciji `__len__` in `__getitem__`. Funkcija `__len__` nam vrne dolžino našega seznama podatkov, funkcija `__getitem__` pa nam vrne slike in pripadajoče oznake.

V našem primeru, smo ga zgradili tako, da vsebuje dva vhodna parametra - pot do naših podatkov (`main_path_to_data`) in seznam naših slik/rezin z oznakami (`data_info`).

```
1 import numpy as np
2 import torch.utils.data as tdata
3 import os
4
5 class DataReader(tdata.Dataset):
6     def __init__(self, main_path_to_data, data_info):
7         super(DataReader, self).__init__()
8         self.data = data_info
9         self.num_sample = len(self.data)
10        self.main_path_to_data = main_path_to_data
11
12    def __len__(self):
13        return self.num_sample
14
```

```

15 def __getitem__(self, n):
16     filename, label = self.data[n]
17     path_to_file = os.path.join(self.main_path_to_data, filename)
18     img = np.load(path_to_file)
19     return img, np.float32([label])

```

Pri tem je `main_path_to_data` mapa v kateri imamo shranjene naše procesirane podatke. V primeru, da delate na Marvinu bo to: `/data/PSUF_naloge/5-naloga/processed`. Drugi argument, `data_info`, pa predstavlja seznam (npr. za učno množico, testno množico ali validacijsko množico), ki je zgrajen sledeče: `[[subpath_to_slice1, oznaka1], ...]`. Oznaka je bodisi 0 (blaga okužba) ali 1 (huda okužba). Učna množica, kjer je izbrana le centralna rezina (slice4 od možnih 10 centralnih rezin slice0, ... slice9), zgleda sledeče:

```

1 [['mild/study_0221_slice_4.npy', 0],
2 ['severe/study_0607_slice_4.npy', 1],
3 ['mild/study_0155_slice_4.npy', 0],
4 ['severe/study_1029_slice_4.npy', 1],
5 ['severe/study_0855_slice_4.npy', 1], ...]

```

Kot vidimo, so podatki naključno porazdeljeni po seznamu (rezultat naključne generacije učne množice). Testna in validacijska množica podatkov, kjer pa nas zanima vseh 10 centralnih rezin, sta urejeni tako, da so v seznamu po vrsti za vsakega pacienta zapisana imena vseh 10 centralnih rezin.

```

1 [['severe/study_0883_slice_0.npy', 1],
2 ['severe/study_0883_slice_1.npy', 1],
3 ['severe/study_0883_slice_2.npy', 1],
4 ['severe/study_0883_slice_3.npy', 1],
5 ['severe/study_0883_slice_4.npy', 1],
6 ['severe/study_0883_slice_5.npy', 1],
7 ['severe/study_0883_slice_6.npy', 1],
8 ['severe/study_0883_slice_7.npy', 1],
9 ['severe/study_0883_slice_8.npy', 1],
10 ['severe/study_0883_slice_9.npy', 1],
11 ['mild/study_0220_slice_0.npy', 0],
12 ['mild/study_0220_slice_1.npy', 0],
13 ['mild/study_0220_slice_2.npy', 0],
14 ['mild/study_0220_slice_3.npy', 0],
15 ['mild/study_0220_slice_4.npy', 0],
16 ['mild/study_0220_slice_5.npy', 0],
17 ['mild/study_0220_slice_6.npy', 0],
18 ['mild/study_0220_slice_7.npy', 0],
19 ['mild/study_0220_slice_8.npy', 0],
20 ['mild/study_0220_slice_9.npy', 0], ...]

```

Če združimo poti `main_path_to_data` in `path_to_slice` (za to uporabimo `os.path.join`), pridemo do zelene rezine, npr. prva izmed 10 centralnih rezin pacienta `study_0883`, se bo nahajala na lokaciji:

```

1 '/data/PSUF_naloge/5-naloga/processed/' + 'severe/study_0883_slice_0.npy'

```

Kot vidimo iz seznama ima oznako 1, kar pomeni, da gre za resno okužbo zaradi COVID-19.

7.2 Nalagalnik podatkov - DataLoader

Ko delamo z veliko količino podatkov vseh slik ne moremo shraniti v seznam, zato v Pytorchu prikladno uporabimo t.i. nalagalnik podatkov - **DataLoader**. Naš model bo tako v pomnilnik shranjeval zgolj toliko slik, kot jih v tistem trenutku namerava procesirati. Poleg tega še definiramo velikost serije (ang. *batch size*), ki nam določa število slik, ki se bodo propagirale skozi model. Za učno množico lahko uporabimo (priporočeno) naključno mešanje (ang. *shuffle*). Za optimizacijo hitrosti pa argumenta `pin_memory` in `num_workers`.

```

1
2 from torch.utils.data import DataLoader
3 from datareader import DataReader
4
5 # DataLoader za ucno mnozico:
6 train_datareader = DataReader(main_path_to_data, train_info)
7 train_generator = DataLoader(train_datareader, batch_size=16, shuffle=True,
8                               pin_memory=True, num_workers=2)
9
10 #DataLoader za testno mnozico:
11 test_datareader = DataReader(main_path_to_data, test_info)
12 test_generator = DataLoader(test_datareader, batch_size=10, shuffle=False, pin_memory
13                             =True, num_workers=2)

```

Če želite uporabljati napisani preprosti skripti za učenje (`train.py`) in testiranje (`test.py`) potem `DataLoader` za validacijo in testiranje ne spreminjajte (`batch_size=10`, `shuffle=False`), `batch_size` je namreč z razlogom nastavljen na 10, saj je za vsakega pacienta v testni in validacijski množici natanko 10 centralnih rezin v aksialni ravnini (prilagodite po potrebi).

7.3 Izgradnja preprostega modela

Naš model definiramo v datoteki `model.py`. Metodologija je analogna `Tensorflow-u`, ki ste ga že spoznali. Plasti nevronske mreže opišemo v funkciji `__init__`. V tem preprostem modelčku imamo zgolj eno konvolucijsko plast, ki sprejme en vhodni kanal (CT rezina velikosti 512x512). Izhodni kanali so štirje, vsakič pa bo uporabljen kernel velikosti 7x7 s korakom 2 v obeh smereh in oblazinjenjem velikosti 3 na obeh straneh. Definiramo še normalizacijo vsake serije (`batch-a`) in dve klasični nevronske plasti. Prva ima na vhodu 65356 nevronov na izhodu 1000, druga pa na vhodu 1000 in izhodu 1.

Za vsak model, ki ga zgradimo sami, moramo napisati še funkcijo za propagacijo v smeri naprej (`forward`).

V tem preprostem primeru v funkciji `forward` uporabimo zaporedoma:

1. Konvolucija (`Conv2d`)
2. Normalizacija serije (`BatchNorm2d`)
3. Aktivacijska funkcija ReLU (`F.relu`)
4. Maksimalni združevalni sloj v 2D (`F.max_pool2d`)
5. Sploščenje rezultata konvolucije v 1D (`x.view(-1, N)`)
6. Klasična linearna nevronska plast (`Linear`)
7. Aktivacijska funkcija ReLU (`F.relu`)
8. Klasična linearna nevronska plast (`Linear`)

Pri tem je dobra praksa, da če želimo kot rezultat našega modela verjetnost za pripadnost enemu izmed dveh razredov (binarna klasifikacija), v zadnjem koraku funkcije `forward` namenoma ne uporabimo sigmoidne funkcije. Moramo pa zato pri izbiri kriterijske funkcije (`criterion`) biti pazljivi in vzeti `BCEWithLogitsLoss`. Slednja namreč uporabi log-sum-exp trik, kar izboljša numerično stabilnost. Če želimo pri napovedi verjetnost (npr. v času validacije ali testiranja) pa še moramo izhod iz funkcije `forward` dodatno spustiti skozi sigmoidno funkcijo (več o tem v nadaljevanju).

```

1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class ModelCT(torch.nn.Module):
6     def __init__(self):
7         super(ModelCT, self).__init__()
8         self.conv1 = nn.Conv2d(1, 4, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
9             bias=False)
10        self.bn1 = nn.BatchNorm2d(4)
11        self.fc1 = nn.Linear(65536, 1000)
12        self.fc2 = nn.Linear(1000, 1)
13
14    def forward(self, x):
15        x = self.conv1(x)
16        x = self.bn1(x)
17        x = F.relu(x)
18        x = F.max_pool2d(x, (2, 2))
19        x = torch.flatten(x, 1)
20        x = F.relu(self.fc1(x))
21        x = self.fc2(x)
22
23        return x

```

7.4 Učenje modela

Za učenje modela uporabimo skripto `train.py`. Glavni koraki za predpripravo učenja so v izbiri kriterijske funkcije, optimizatorja in opcijsko tudi urnika spreminjanja stopnje učenja (ang. *scheduler*) - ter seveda vseh pripadajočih hiperparametrov (stopnja učenja, regularizacija, ...).

```

1 import torch
2
3 # Preverimo ali imamo na voljo gpu, ce ne uporabimo cpu
4 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
5
6 # Model posljemo na razpoložljivo enoto
7 model = ModelCT()
8 model.to(device)
9
10 # Kriterijska funkcija
11 criterion = torch.nn.BCEWithLogitsLoss()
12
13 # Izbira optimizatorja Adam
14 optimizer = torch.optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
15
16 # Definicija urnika spreminjanja stopnje učenja -> LR = LR*multiplicator
17 scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, multiplicator,
18     last_epoch=-1)

```

Pomembno je, da pred pričetkom učenja naš model postavimo v stanje učenja, s klicem `model.train()`. Iterator `train_generator` nam generira batche podatkov, s katerimi nato v propagaciji v smeri naprej izračunamo izgubo ter s to informacijo v propagaciji v smeri nazaj popravimo uteži našega modela.

```

1 for epoch in range(total_epoch):
2     model.train() # nastavimo model v fazo ucenja
3
4     for item_train in train_generator: # izberemo generiran batch
5         # Nalozimo slike batcha (x) in oznake (y), in jih posljemo na gpu ali cpu
6         x, y = item_train
7         x = x.to(device)
8         y = y.to(device)
9
10        # Propagacija naprej in izracun izgube
11        optimizer.zero_grad()
12        y_hat = model.forward(x)
13        loss = criterion(y_hat, y)
14
15        # Propagacija nazaj
16        loss.backward()
17        optimizer.step()
18
19    scheduler.step() # sprememba urnika za stopnjo ucenja vsak epoch
20    torch.save(model.state_dict(), 'trained_model_weights.pth') # shranimo model

```

7.5 Testiranje modela

Pri testiranju moramo naš model, ki smo ga naučili (uteži) naložiti. Pred pričetkom testiranja, ga moramo obvezno nastaviti v stanje `.eval()`, saj s tem zaklenemo morebitne dropout plasti in normalizacijo batch-ov. Ne pozabimo, da želimo napovedati verjetnosti za 10 centralnih rezin, zato imamo v `test_generator` nastavljen parametra `batch_size=10` in `shuffle=False`. S tem dosežemo, da v vsakem batch-u dobimo 10 centralnih rezin enega pacienta. Če bi npr. želeli našo končno verjetnost za pripadnost enemu izmed dveh razredov (huda/blaga okužba) podati kot povprečje verjetnosti preko vseh 10 centralnih rezin, bi naša koda izgledala nekako tako:

```

1 import torch
2 from sklearn.metrics import roc_auc_score
3
4 model = ModelCT()
5 model.to(device)
6 model.load_state_dict(torch.load('trained_model_weights.pth'))
7 model.eval()
8
9 predictions = []
10 trues = []
11
12 for item_test in test_generator:
13     # Nalozimo slike batcha (x) in oznake (y), in jih posljemo na gpu ali cpu
14     x, y = item_test
15     x = x.to(device)
16     y = y.to(device)
17
18     # Propagacija naprej
19     with torch.no_grad():
20         y_hat = model.forward(x)
21         y_hat = torch.sigmoid(y_hat) # Ne pozabite sigmoidne funkcije pri testiranju.
22
23     # Končna napoved za batch je povprečje vseh 10 rezin (numpy deluje le na cpu)
24     predictions.append(np.mean(y_hat.cpu().numpy()))
25     # V trues shranimo pravo oznako (ground-truth)
26     trues.append(y.cpu().numpy()[0])
27
28 # Izracunamo AUC
29 auc = roc_auc_score(trues, predictions)

```

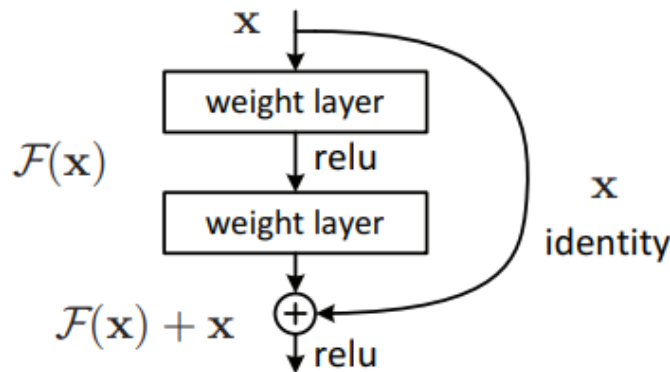
7.6 Modifikacija modela ResNet18

Pytorch omogoča tudi zelo preprosto implementacijo kompleksnih konvolucijskih nevronske mreže, kot so npr. AlexNet, GoogleNet, VGG in ResNet. Prvotni cilj teh mrež je bila klasifikacija slik iz baze ImageNet v 1000 različnih razredov. To pa ne pomeni, da z ustrezno modifikacijo arhitekturnih zasnov teh mrež ne moremo uporabiti tudi na drugih problemih. V tej nalogi se bomo posvetili mreži iz družine ResNet, ki ima 18 plasti - ResNet18. Ta mreža je najbolj preprosta, obstajajo še globlje z 34, 50, 101 in 152 plastmi (Slika 10).

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

Slika 10: Prikaz različno globokih ResNet arhitektur za klasifikacijo slik iz baze ImageNet. Povzeto iz [18].

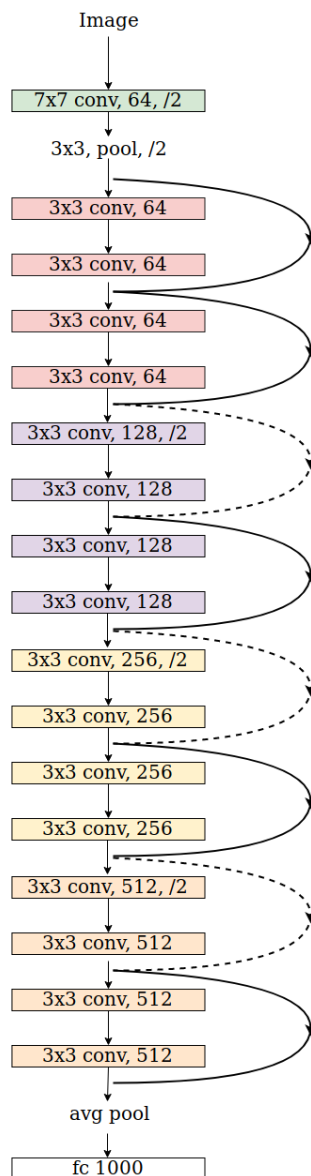
Večanje globine in števila konvolucijskih plasti omogoča odkrivanje kompleksnejših značilnosti iz vhodnih slik. Pri tem se z večanjem globine modela lahko pojavijo tudi težave, in sicer predvsem s t.i. izginjajočimi (ang. *vanishing*) gradienti in poslabšanjem rezultata celo na učni množici. Rešitev tega problema predstavljajo t.i. ResNet arhitekture [18], kjer se rezultat i -te konvolucije prenese na vhod $(i+N)$ -te. N pri tem predstavlja neko manjšo naravno število (za ResNet18 je $N=3$). Če je rezultat i -te konvolucijske plasti x , in $F(x)$ rezultat konvolucij plasti z indeksi $i+k$, kjer $k \in (1, \dots, N-1)$, potem je vhod na $(i+N)$ -to plast $\text{ReLU}(F(x) + x)$, pri čemer ReLU predstavlja aktivacijsko funkcijo (Slika 11).



Slika 11: Rezidualno učenje, osnovni blok. Povzeto iz [18].

7.6.1 Pytorch implementacija modificiranega modela ResNet18

Mi si bomo po navdihu članka iz revije Nature Medicine: *Artificial intelligence-enabled rapid diagnosis of patients with COVID-19* [17] ogledali modifikacijo mreže ResNet18 (Slika 12), ki je bila implementirana za diagnozo pacientov s COVID-19 okužbo. Gre za podoben problem kot v tej nalogi, le da mi ločujemo med blago in resno COVID-19 okužbo.



Slika 12: Arhitekturna zasnova modela ResNet18 za klasifikacijo 1000 različnih slik iz baze ImageNet. Povzeto iz [18].

Kot bomo videli je prednost Pytorch predvsem v tem, da lahko kompleksne arhitekture, kot je npr. ResNet18, zelo preprosto modificiramo za kak drug problem. Hkrati lahko zelo enostavno uporabimo uteži splošnih filtrov začetnih konvolucijskih plasti, ki so bile naučene na milijonih slik iz baze ImageNet. Te plasti lahko zaklenemo in mrežo uporabimo kot ekstraktor značilk, lahko pa jih pustimo odklenjene in jih samo do-učimo za naš problem (ang. *fine-tuning*). Slednje bomo naredili tudi v tej nalogi, saj se slike iz baze ImageNet zelo razlikujejo od našega problema (z globino mreže filtri postajajo vedno bolj odvisni od učne množice, zato ne bi imelo smisla zakleniti vseh plasti). Poglejmo si, kako to naredimo v praksi.

Najprej moramo v zbirki modelov `torchvision.models` poiskati arhitekturo ResNet18. Pri tem s parametrom `pretrained` določimo, ali želimo že naučen model na bazi slik ImageNet ali ne.

Opomba: Pri novejših verzijah Pytorcha se namesto parametra `pretrained` uporablja bolj fleksibilen parameter `weights`, ki ga nastavimo na `ResNet18.Weights.IMAGENET1K_V1`.

```
1 import torchvision.models as tmodels
2
3 backbone = tmodels.resnet18(pretrained=True)
```

Tukaj se moramo zavedati, da je ImageNet baza barvnih slik, tako da so vsi modeli optimizirani za 3 vhodne kanale (R, G, B). Torej `conv1` v nespremenjeni ResNet18 mreži zgleda tako:

```
1 nn.Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
```

Ta problem rešimo s tem, da vhodno konvolucijsko plast spremenimo. Naš model bo namreč na vходу prejel le en kanal in ne treh (pozor, s tem izgubimo uteži filtrov vhodnih plasti iz naučenega modela, ali obstaja še kak drug način?).

```
1 backbone.conv1 = nn.Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
    bias=False)
```

Od končnega modela nas loči zgolj implementacija funkcije `forward`.

```
1 import torch
2 import torch.nn as nn
3 import torchvision.models as tmodels
4
5
6 class ModelCT(nn.Module):
7     def __init__(self):
8         super(ModelCT, self).__init__()
9         # Definiramo ogrodje za model
10        self.backbone = tmodels.resnet18(pretrained=True)
11
12        # Spremenimo prvo konvolucijsko plast ogrodja
13        self.backbone.conv1 = nn.Conv2d(1, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3),
14            bias=False)
15
16        # Dodamo logiko zadnjih plasti , povzeto iz clanka [17]
17        self.convolution2d = nn.Conv2d(512, 1, kernel_size=(1, 1), stride=(1, 1), bias=True)
18        self.fc_maxpool = nn.AdaptiveMaxPool2d((1, 1))
19
20    def forward(self, x):
21        x = self.backbone.conv1(x) # Spremenjena plast za 1 vhodni kanal
22        x = self.backbone.bn1(x) # nespremenjeno
23        x = self.backbone.relu(x) # nespremenjeno
24        x = self.backbone.maxpool(x) # nespremenjeno
25        x = self.backbone.layer1(x) # conv_2x iz Slike 10
26        x = self.backbone.layer2(x) # conv_3x iz Slike 10
27        x = self.backbone.layer3(x) # conv_4x iz Slike 10
28        x = self.backbone.layer4(x) # conv_5x iz Slike 10
29
30        # Nasa modifikacija zadnjih plasti:
31        x = self.convolution2d(x)
32        x = self.fc_maxpool(x)
33
34        x = torch.flatten(x, 1)
35        return x
```

Argumentacijo za izbrano modifikacijo zadnjih plasti mreže ResNet18 lahko najdete v dodatku omenjenega članka [17]. Če bi želeli naš model naučiti od začetka, bi preprosto spremenili parameter `pretrained` iz `True` v `False` (oz. v novejših verzijah Pytorcha primerno prilagodili parameter `weights`).

8 Navodilo naloge

Na voljo imate učno množico `train_info`, kjer se nahajajo imena centralnih rezin 60% vseh pacientov, označenih z oznako 0 (**blaga okužba**) in 1 (**resna okužba**). S pomočjo opazovanja funkcije izgube med učenjem modela in opazovanjem vrednosti AUC na validacijski množici `valid_info` poskusite najti optimalne hiperparametre, ki vodijo do uteži modela, ki bo na neodvisni testni množici `test_info` po vašem mnenju dosegel najboljši rezultat. Vse slike imate že procesirane (mapa `processed`). To naredite za 3 različne modele:

1. Vaša konvolucijska nevronska mreža.
2. Model z modificirano arhitekturo ResNet18 in ne-naučenimi filtri na bazi ImageNet.
3. Model z modificirano arhitekturo ResNet18 in naučenimi filtri na bazi ImageNet.

Arhitekturo za vašo konvolucijsko nevronska mrežo napišite sami (spremenite funkcijo `forward` v skripti `model.py`), modela (2) in (3) pa imate že na voljo. Pri modelih (2) in (3) ne zaklepajte plasti (pustite vse tako kot je) in dovolite, da se vaš model do-uči na vaši učni množici (*fine-tuning*). Vse skupaj je pripravljeno tako, da načeloma lahko le zaženete skripto `run.py` v kateri v slovar `hyperparameters` nastavite vrednosti zelenih hiperparametrov.

Za vsako izmed treh arhitektur izberite uteži modela, ki se bodo po vašem mnenju najbolj obnesle na testni množici. Ko boste za vsako izmed treh arhitektur imeli izbrane uteži, na testni množici naredite sledeče:

1. Izračunajte AUC
2. Izberite prag (ang. *threshold*) in argumentirajte svojo odločitev
3. Izračunajte točnost (ang. *accuracy*)

Na koncu primerjajte rezultate iz vseh 3 arhitektur (en rezultat na arhitekturo).

8.1 Primer uporabe skripte run.py

Zaenkrat je končna verjetnost za pripadnost razredu 0 ali 1 za validacijsko in testno množico definirana kot povprečje napovedi za 10 centralnih rezin (napoved je izračunana za vsako rezino posebej, končna verjetnost pa podana kot povprečje teh 10 napovedi).

```
1 import os
2 import json
3 from model import ModelCT
4 from train import Training
5 from test import Testing
6
7 if __name__ == '__main__':
8
9     main_path_to_data = '/data/PSUF_naloga/5-naloga/processed'
10    path_to_model = "trained_models/ime_mape_za_model"
11    # Nalozimo sezname za ucno, validacijsko in testno mnozico
12    with open(os.path.join(main_path_to_data, "train_info.json")) as f:
13        train_info = json.load(f)
14    with open(os.path.join(main_path_to_data, "val_info.json")) as f:
15        valid_info = json.load(f)
16    with open(os.path.join(main_path_to_data, "test_info.json")) as f:
17        test_info = json.load(f)
18
19    # Izberemo model
20    my_model = ModelCT()
21
22    # Nastavimo hiperparametre v slovarju
23    hyperparameters = {}
24    hyperparameters['learning_rate'] = 0.2e-3 # learning rate
25    hyperparameters['weight_decay'] = 0.0001 # weight decay
26    hyperparameters['total_epoch'] = 20 # total number of epochs
27    hyperparameters['multiplicator'] = 0.95 # each epoch: LR=LR*multiplicator
28
29    # Ustvarimo ucni in testni razred
30    TrainClass = Training(main_path_to_data)
31    TestClass = Testing(main_path_to_data)
32
33    # Naucimo model za izbrane hiperparametre
34    aucs, losses = TrainClass.train(train_info, valid_info, my_model, hyperparameters,
35                                   path_to_model)
36
37    # Utezi naucenega modela
38    path_to_model_weights = os.path.join(path_to_model, "trained_model_weights.pth")
39
40    # Testiramo nas model na testni mnozici
41    auc, fpr, tpr, thresholds, trues, predictions = TestClass.test(test_info,
42                                                                     my_model, path_to_model_weights)
```

Iz funkcije `TrainClass.train(...)` boste dobili:

- `aucs` - kako se AUC spreminja na validacijski množici skozi proces učenja
- `losses` - kako se spreminja izguba na učni množici skozi proces učenja

Iz funkcije `TestClass.test(...)` boste dobili:

- `auc` - AUC na testni množici
- `fpr` - False Positive Rate na testni množici
- `tpr` - True Positive Rate na testni množici

- **thresholds** - pragovi za določitev AUC
- **true**s - dejanski razredi
- **predictions** - napovedi modela - verjetnosti med 0 in 1 za pripadnost razredu 1 (huda okužba)

Literatura

- [1] Hellewell J, Abbott S, Gimma A, et al (2020) Feasibility of controlling COVID-19 outbreaks by isolation of cases and contacts. *Lancet Glob Heal* 8:e488–e496. [https://doi.org/10.1016/S2214-109X\(20\)30074-7](https://doi.org/10.1016/S2214-109X(20)30074-7)
- [2] Miller IF, Becker AD, Grenfell BT, Metcalf CJE (2020) Disease and healthcare burden of COVID-19 in the United States. *Nat Med* 26:1212–1217. <https://doi.org/10.1038/s41591-020-0952-y>
- [3] World Health Organization (2020) Use of chest imaging in COVID-19. <https://www.who.int/publications/i/item/use-of-chest-imaging-in-covid-19>. Accessed 13 Aug 2020
- [4] Bernheim A, Mei X, Huang M, et al (2020) Chest CT Findings in Coronavirus Disease-19 (COVID-19): Relationship to Duration of Infection. *Radiology* 295:200463. <https://doi.org/10.1148/radiol.2020200463>
- [5] Prokop M, van Everdingen W, van Rees Vellinga T, et al (2020) CO-RADS – A categorical CT assessment scheme for patients with suspected COVID-19: definition and evaluation. *Radiology* 201473. <https://doi.org/10.1148/radiol.2020201473>
- [6] Zu ZY, Jiang M Di, Xu PP, et al (2020) Coronavirus Disease 2019 (COVID-19): A Perspective from China. *Radiology* 200490. <https://doi.org/10.1148/radiol.2020200490>
- [7] Ai T, Yang Z, Hou H, et al (2020) Correlation of Chest CT and RT-PCR Testing in Coronavirus Disease 2019 (COVID-19) in China: A Report of 1014 Cases. *Radiology* 200642. <https://doi.org/10.1148/radiol.2020200642>
- [8] Xie X, Zhong Z, Zhao W, et al (2020) Chest CT for Typical 2019-nCoV Pneumonia: Relationship to Negative RT-PCR Testing. *Radiology* 200343. <https://doi.org/10.1148/radiol.2020200343>
- [9] Colombi D, Bodini FC, Petrini M, et al (2020) Well-aerated Lung on Admitting Chest CT to Predict Adverse Outcome in COVID-19 Pneumonia. *Radiology* 201433. <https://doi.org/10.1148/radiol.2020201433>
- [10] Pan F, Ye T, Sun P, et al (2020) Time Course of Lung Changes at Chest CT during Recovery from Coronavirus Disease 2019 (COVID-19). *Radiology* 295:715–721. <https://doi.org/10.1148/radiol.2020200370>
- [11] Li W, Cui H, Li K, et al (2020) Chest computed tomography in children with COVID-19 respiratory infection. *Pediatr Radiol* 50:796–799. <https://doi.org/10.1007/s00247-020-04656-7>
- [12] Morozov, S. Andreychenko AB et al. (2020) MosMedData: Chest CT Scans with COVID-19 Related Findings, 2020, v. 1.0., https://mosmed.ai/datasets/covid19_1110
- [13] Prikaz CT značilnk. www.sl.wikipedia.org/wiki/Tkivo (22.11.2020).
- [14] Prikaz CT naprave. www.siemens.com (22.11.2020).
- [15] Prikaz CT ravnin. <https://www.radiologycafe.com/> (22.11.2020).

- [16] Rekonstrukcija CT slike. <https://link.springer.com/article/10.1007/s00247-014-3102-1> (22.11.2020).
- [17] Mei, Xueyan, et al. "Artificial intelligence-enabled rapid diagnosis of patients with COVID-19." *Nature Medicine* (2020): 1-5.
- [18] He, Kaiming, et al. "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.